

Why the GOTHAM PDF Files Broke

A shared-buffer bug with a very specific fingerprint

GOTHAM PDF reconstruction notes

June 2026

Status: Strongly supported root cause. A same-state broken-line/fixed-line AthenaK reproduction has not yet been run.

1 What we are trying to explain

Fourteen multidimensional PDF stream definitions were used during the five GOTHAM simulations. The corruption summary reports that, at the checked latest snapshots, twelve are unusable for radial or angular diagnostics. One, `output29`, keeps a trustworthy angle-integrated radial profile but not its angular distribution. One, `output31`, remains useful. The attempted full archive audit failed before writing its final CSV, so the archive-wide scope is not independently preserved as a completed artifact.

That mixed outcome is the question. A vague “the PDFs are corrupted” story is not enough. We need a mechanism that predicts *which* axes fail, *which* marginals survive, and why total weight can remain nearly right while the joint distribution is badly wrong.

2 The simple picture

Here is the simple version. AthenaK computed several derived coordinates into one shared array. The `coord_abcscostheta` routine resized that array after earlier coordinates had already been written. In Kokkos, `realloc` does not promise to preserve the old contents. The new angle was written. Earlier rows, including radius in the affected orderings, became unsafe. In the affected archive, unsafe radius behaves like zero and lands in radial underflow; unsafe `coord_abcscostheta` behaves like zero and lands in its first interior bin.

The simulation state was fine. The histogram formulas were mostly fine. The temporary coordinate workspace was not.

3 The cartoon version

Read this left to right. Before the angle routine runs, the shared workspace already contains rows written by earlier coordinate routines. The inner `realloc` replaces that storage, shrinks its first extent from $n_{\text{mb,max}}$ to n_{mb} , and the angle routine fills back only its own row.

Now comes the part that explains `output29`. Its next coordinate is radius. At the start of that call, the common capacity check sees $n_{\text{mb}} < n_{\text{mb,max}}$, reallocates the workspace back to $n_{\text{mb,max}}$, invalidates the

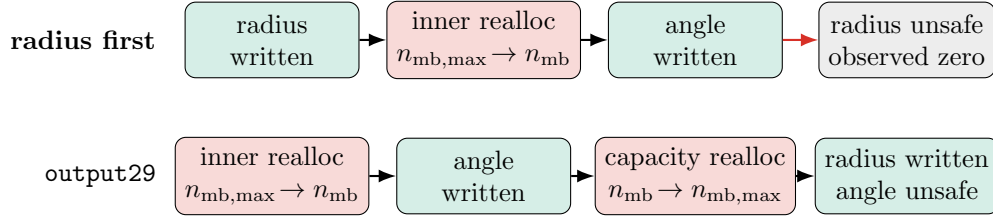


Figure 1: The bad angle call causes one destructive reallocation in radius-first products and triggers a second capacity reallocation before radius in `output29`. The coordinate written last survives. *This is a schematic, not evidence.*

angle row, and then writes radius. So the two orders fail differently:

$$\begin{aligned} \text{radius first: } r &\longrightarrow \text{inner realloc} \longrightarrow |\cos \theta| \Rightarrow r \text{ lost,} \\ \text{angle first: } |\cos \theta| &\longrightarrow \text{capacity realloc} \longrightarrow r \Rightarrow |\cos \theta| \text{ lost.} \end{aligned}$$

The annoying part is that the file writer can still count every cell. It just counts cells using a coordinate that has been replaced by zeros. So a total can look sensible while a coordinate axis has collapsed.

4 The more precise version

For a PDF product p , each cell c contributes to one flattened bin:

$$H_p(b) = \sum_c \Delta V_c w_p(c) \mathbf{1}[B_p(\mathbf{q}_c) = b].$$

The bug changes the coordinate vector $\mathbf{q}_c$ used by the bin map B_p . For this fourteen-product catalog, the final weight rows survive: mass and volume live outside the shared coordinate workspace, while `mdot_sph` and `edot_sph` are computed after the destructive calls. That distinction predicts the observed split:

- totals can remain correct because the same weights are accumulated;
- a marginal over the coordinate written last can survive;
- the full joint distribution fails where the erased coordinate chooses the wrong bins.

The source history gives us an unusually clean before-and-after. Commit `ec009a27` changed the guarded allocation into an unconditional allocation inside `coord_abcscostheta`; commit `db953863` removed it. The shared array is already sized near the beginning of `ComputeDerivedVariable()`. Full hashes are recorded in the traceability notes.

5 What this idea predicts

A root-cause story is only useful if it gets the exceptions right. This one has four places where it could have failed.

Prediction	Diagnostic	Existing result
Radius-first streams collapse into radius underflow	Streaming sparse-shard radius audit	Reported at checked latest snapshots across all five simulations; completed machine-readable audit artifact missing.
<code>output29</code> keeps its radial marginal but loses angle	Compare rebuilt and archived full 2D distributions and radial marginals	Full $E_{L1} = 1.92$; radial-marginal $E_{L1} = 1.60 \times 10^{-11}$.
<code>output31</code> stays intact because it does not use <code>coord_abcscostheta</code>	Compare matched eight-shard rebuilt and archived 4D controls	Matched eight-shard full-4D $E_{L1} = 3.87 \times 10^{-7}$.
Weights remain counted even when coordinates are wrong	Compare signed totals	Control totals agree at roughly 10^{-12} normalized error.

6 What the evidence actually shows

What to look at. Start with the lower-left panel. The archived and rebuilt radial curves sit on top of each other. Now move to the lower-right panel: the absolute-weight angular-support marginals separate sharply. The radial agreement tells us the source state and radial accounting survived. The failure appears only when angle matters.

Caveat. This is an eight-shard matched-subset comparison, not a complete 1024-shard reconstruction. It tests the corruption signature and numerical repair path. It does not certify global AMR completeness.

What to look at. The red bar is not a failure of the reconstruction. It is the required negative control. A reducer that exactly reproduced the full archived `output29` would have reproduced the broken angle. The green bars show that the same reducer agrees closely with controls that should be intact.

Caveat. The dashed 2×10^{-3} line is the full-`output31` validator limit only. The other controls use tighter, product-specific limits.

7 The key figures

The cartoon explains the mechanism. The paired radial-and-angular comparison is the evidence that makes it convincing. Exact inputs and metrics are recorded in the root-level `figure_metrics.json` ledger.

8 What works

This story earns confidence because it explains the exceptions, not just the failures:

- it identifies the exact source operation;
- it explains why twelve streams collapse in radius;
- it explains why `output29` has a salvageable radial marginal;

The diagnostic fingerprint: total and radial transport survive; angle does not

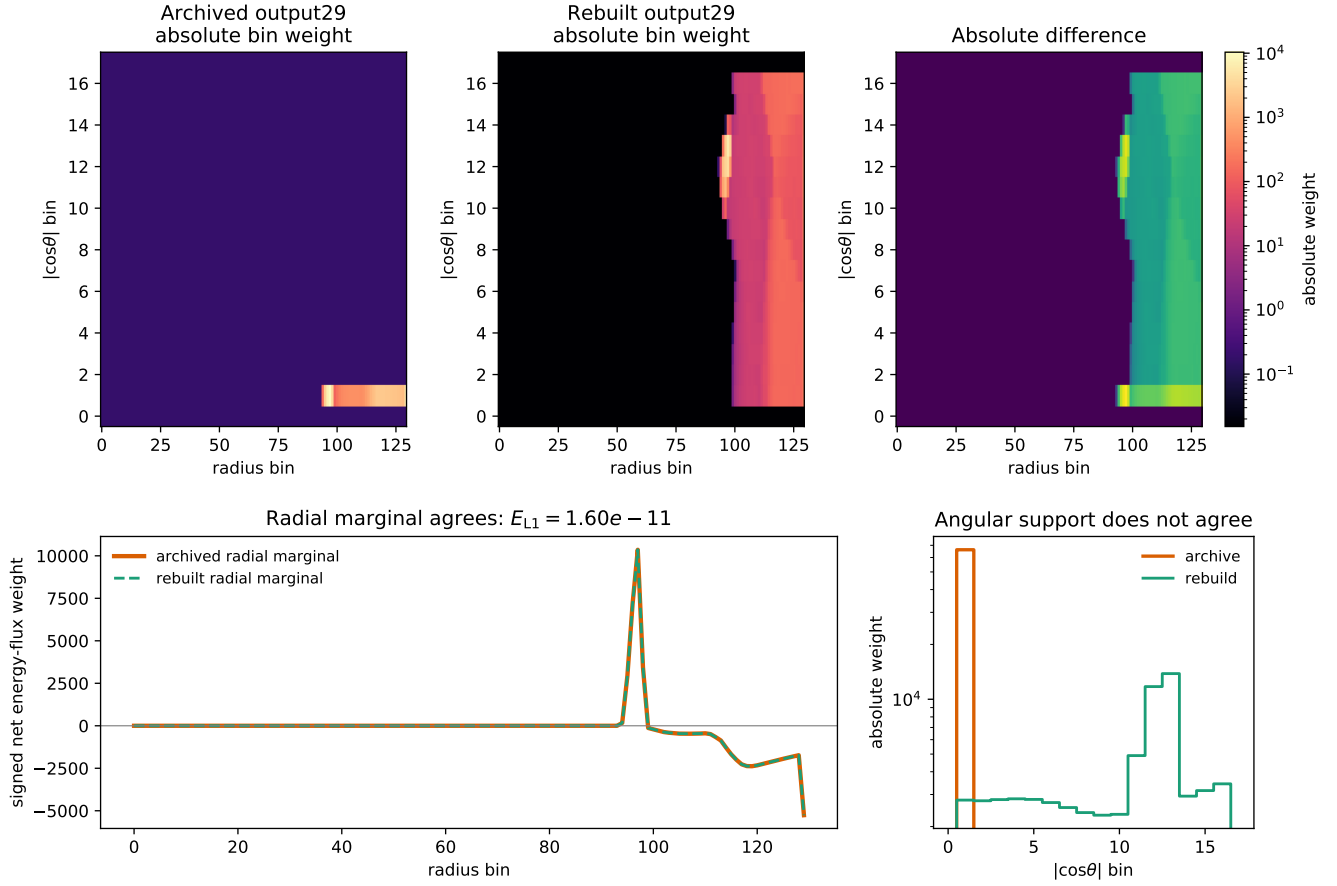


Figure 2: Eight-shard comparison at exact archived PDF time 10.520002963096713. Inputs are the rebuilt dense output29 and the matching first eight archived sparse shards. Generated by docs/explanatory_writeups/scripts/generate_figures.py.

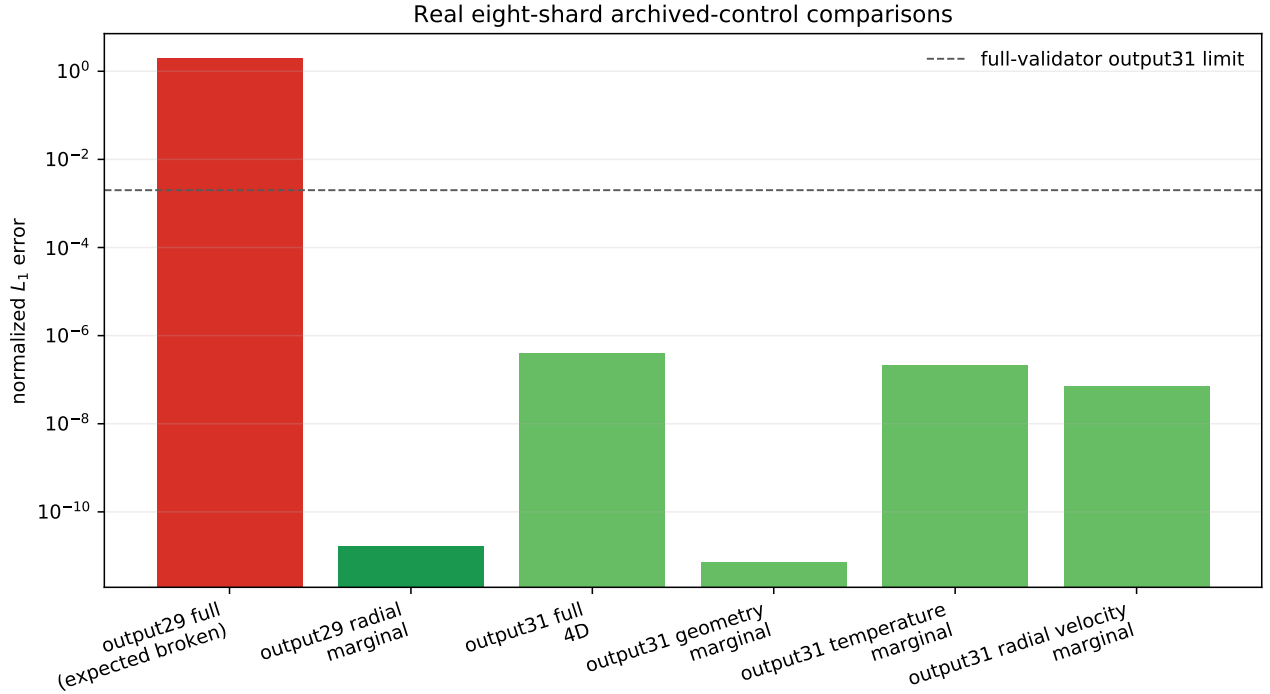


Figure 3: Normalized L_1 errors for the same eight-shard real-data comparison. Green bars are quantities expected to survive the archive bug. The red bar is the broken full archived `output29`.

- it explains why `output31` is a useful negative control for this specific bug;
- it predicts the repaired-angle disagreement without requiring lost simulation state.

9 What does not work

The archive cannot be fixed by relabeling bins or renormalizing totals. Once a joint coordinate was erased before binning, the missing association between cell weight and angle is gone from that PDF file. The twelve broken streams need reconstruction from the saved full-volume AMR outputs.

Also, “all PDFs are broken” is too strong. `output31` remains useful, and the angle-integrated radial marginal of `output29` remains useful. Those exceptions are part of the evidence, not inconvenient details.

10 How this could be wrong

A skeptical alternative would be a reader bug, a malformed sparse format, or bad simulation state. Those alternatives do not explain why one marginal survives while the joint distribution fails. Still, the source diff plus this fingerprint is not a unique causal intervention: another angle-specific writer bug could imitate part of the same result. The production reader successfully reads the intact stream. The source state reconstructs both intact controls and the preserved marginal. The source diff contains an operation with exactly the expected destructive semantics.

The leading root cause is strong enough to guide the repair. The cleanest remaining causal test is to run the same small AthenaK state with the bad line present and absent and show that the mixed pattern

appears and disappears.

11 What would decide the issue

For the existing archive, we know enough to act. To close the causal loop, run a same-state bad-line/fixed-line A/B test with $n_{\text{mb}} < n_{\text{mb,max}}$. The regression must include both coordinate orders, a trailing derived weight, and full-histogram comparison against an oracle. Merely checking for nontrivial population can miss one of the two reallocation paths. A post-write invariant that flags an unexpected 100% underflow coordinate would catch the observed failure even earlier.

12 Bottom line

Bottom line. The leading explanation is that one derived-coordinate calculation reallocated shared coordinate storage twice in the mixed `output29` case and once in the radius-first cases. The strongest evidence is not merely the source diff. It is the source diff plus the peculiar fingerprint it predicts: totals survive, one radial marginal survives, the angular joint distribution fails, and the negative control remains intact. A same-state old-line/new-line reproduction would upgrade this from strongly supported to demonstrated.

A Revision notes

This document was revised after solution-advocate, logic/technical, evidence/figure, red-team, visual-design, human-editor, and reproducibility checks. Pass two added the second $n_{\text{mb}} \rightarrow n_{\text{mb,max}}$ reallocation, weakened the causal status pending an A/B writer test, separated the eight-shard evidence from the incomplete archive-wide audit, and relabeled `output31` as a differential negative control. Review records live in the local `notes/` directory.